

Chapter 4 Operations on Data

- List the three categories of operations performed on data.
- Perform unary and binary logic operations on bit patterns.
- Distinguish between logic shift operations and arithmetic shift operations.
- Perform logic shift operations on bit patterns.
- Understand some applications of logic and shift operations such as setting, unsetting, and flipping bits.
- Perform arithmetic shift operation on integers when they are stored in two's complement format.
- Perform addition and subtraction operations on reals when they are stored in sign and magnitude format.

4-1 LOGIC OPERATIONS

In Chapter 3 we discussed the fact that data inside a computer is stored as patterns of bits.

- Logic operations refer to those operations that apply the same basic operation on individual bits of a pattern, or on two corresponding bits in two patterns.
- This means that we can define logic operations at the bit level and at the pattern level.
- A logic operation at the pattern level is n logic operations, of the same type, at the bit level where n is the number of bits in the pattern.

4.1.1 Logic operations at bit level

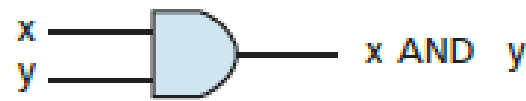
- A bit can take one of the two values: 0 or 1.
- If we interpret **0** as the value *false* and **1** as the value *true*, we can apply the operations defined in Boolean algebra to manipulate bits.
- **Boolean algebra**, named in honor of George Boole, belongs to a special field of mathematics called logic. Boolean algebra and its application to building logic circuits in computers are briefly discussed in Appendix E.
- In this section, we show briefly four bit-level operations that are used to manipulate bits: **NOT**, **AND**, **OR**, and **XOR**.
- Figure 4.1 shows the symbols for these four bit-level operators and their truth tables.
- **Truth table** defines the values of output for each possible input or inputs.

Figure 4.1 *Logic operations at the bit level*



NOT

x	NOT x
0	1
1	0



AND

x	y	x AND y
0	0	0
0	1	0
1	0	0
1	1	1

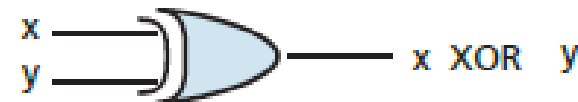
AND	0	1
0	0	0
1	0	1



OR

OR	0	1
0	0	1
1	1	1

x	y	x OR y
0	0	0
0	1	1
1	0	1
1	1	1



XOR

x	y	x XOR y
0	0	0
0	1	1
1	0	1
1	1	0

XOR	0	1
0	0	1
1	1	0

NOT

- The NOT operator is a unary operator: it takes only **one input**.
- The output bit is the complement of the input.

x	NOT x
0	1
1	0

AND

- The AND operator is a binary operator: it takes **two inputs**.
- The output bit is 1 if both inputs are 1s and the output is 0 in the other three cases.

x	AND	0	1	x
0		0	0	0
1		0	1	1
		$x \text{ AND } 0 = 0$	$x \text{ AND } 1 = x$	$x \text{ AND } x = x$

OR (inclusive-or)

- The OR operator is a binary operator: it takes **two inputs**.
- The output bit is 0 if both inputs are 0s and the output is 1 in other three cases.

x	OR	0	1	x
0		0	1	0
1		1	1	1
		$x \text{ OR } 0 = x$	$x \text{ OR } 1 = 1$	$x \text{ OR } x = x$

XOR (exclusive-or)

- The XOR operator is a binary operator like the OR operator, with only one difference: the output is 0 if both inputs are 1s.

x	XOR	0	1	x
0		0	1	0
1		1	0	0
		$x \text{ XOR } 0 = x$	$x \text{ XOR } 1 = \text{NOT}x$	$x \text{ XOR } x = 0$

Example 4.2 The XOR operator is not actually a new operator. We can always simulate it using the other three operators. The following two expressions are equivalent

$$x \text{ XOR } y \leftrightarrow [x \text{ AND } (\text{NOT } y)] \text{ OR } [(\text{NOT } x) \text{ AND } y]$$

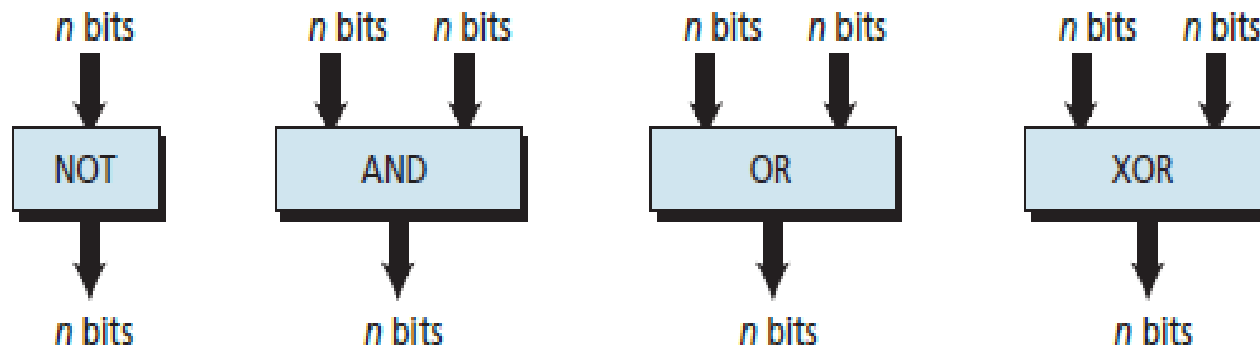
- Prove by truth table

x	y	$x \text{ XOR } y$	$\text{NOT } y$	$\text{NOT } x$	$x \text{ AND } (\text{NOT } y)$	$(\text{NOT } x) \text{ AND } y$	$[x \text{ AND } (\text{NOT } y)] \text{ OR } [(\text{NOT } x) \text{ AND } y]$
0	0	0	1	1	0	0	0
0	1	1	0	1	0	1	1
1	0	1	1	0	1	0	1
1	1	0	0	0	0	0	0

4.1.2 Logic operations at pattern level

- The same four operators (NOT, AND, OR, and XOR) can be applied to an n -bit pattern.
- The effect is the same as applying each operator to each individual bit for NOT and to each corresponding pair of bits for other three operators.
- 4.2 shows these four operators with input and output patterns.

Figure 4.2 *Logic operators applied to bit patterns*



Example 4.3 Use the NOT operator on the bit pattern 10011000.

NOT	1	0	0	1	1	0	0	0	Input
	0	1	1	0	0	1	1	1	Output

Example 4.4 Use the AND operator on the bit patterns 10011000 and 00101010.

	1	0	0	1	1	0	0	0	Input 1
AND	0	0	1	0	1	0	1	0	Input 2
	0	0	0	0	1	0	0	0	Output

Example 4.5 Use the OR operator on the bit patterns 10011001 and 00101110.

	1	0	0	1	1	0	0	1	Input 1
OR	0	0	1	0	1	1	1	0	Input 2
	1	0	1	1	1	1	1	1	Output

Example 4.6 Use the XOR operator on the bit patterns 10011001 and 00101110.

	1	0	0	1	1	0	0	1	Input 1
XOR	0	0	1	0	1	1	1	0	Input 2
	1	0	1	1	0	1	1	1	Output

Applications

The four logic operations can be used to modify a bit pattern.

- Complementing (NOT)
- Unsetting (AND)
- Setting (OR)
- Flipping (XOR)

Example 4.7 Use a mask to **unset (clear)** the five leftmost bits of a pattern. Test the mask with the pattern 10100110.

	1	0	1	0	0	1	1	0	Input
AND	0	0	0	0	0	1	1	1	Mask
	0	0	0	0	0	1	1	0	Output

Example 4.8 Use a mask to **set** the five leftmost bits of a pattern. Test the mask with the pattern 10100110.

	1	0	1	0	0	1	1	0	Input
OR	1	1	1	1	1	0	0	0	Mask
	1	1	1	1	1	1	1	0	Output

Example 4.9 Use a mask to **flip** the five leftmost bits of a pattern. Test the mask with the pattern 10100110.

	1	0	1	0	0	1	1	0	Input 1
XOR	1	1	1	1	1	0	0	0	Mask
	0	1	0	1	1	1	1	0	Output

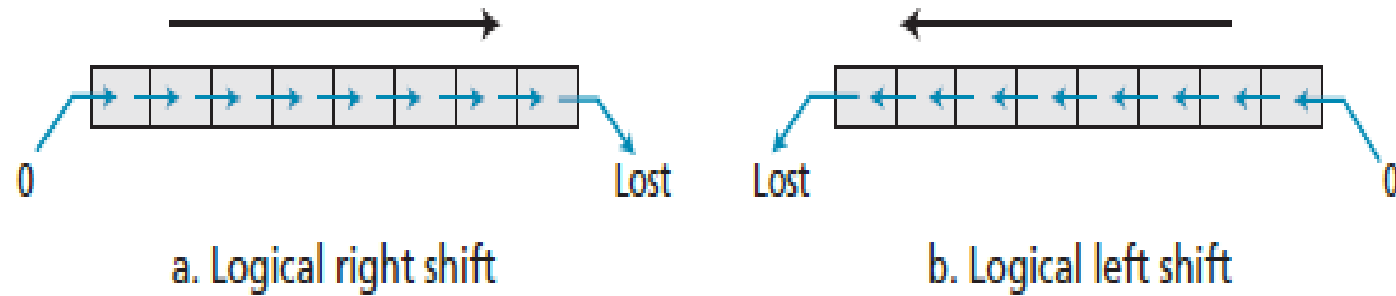
4-2 SHIFT OPERATIONS

- Shift operations move the bits in a pattern, changing the positions of the bits. They can move bits to the left or to the right.
- We can divide shift operations into two categories:
 - logical shift operations
 - arithmetic shift operations.

4.2.1 Logical shift operations

- A logical shift operation is applied to a pattern that does not represent a signed number.
- The reason is that these shift operation may change the sign of the number that is defined by the leftmost bit in the pattern.
- We distinguishes two types of logical shift operations, as described below:
 - Logical shift
 - Logical circular shift (Rotate)

Figure 4.3 Logical shift operations



Example 4.10

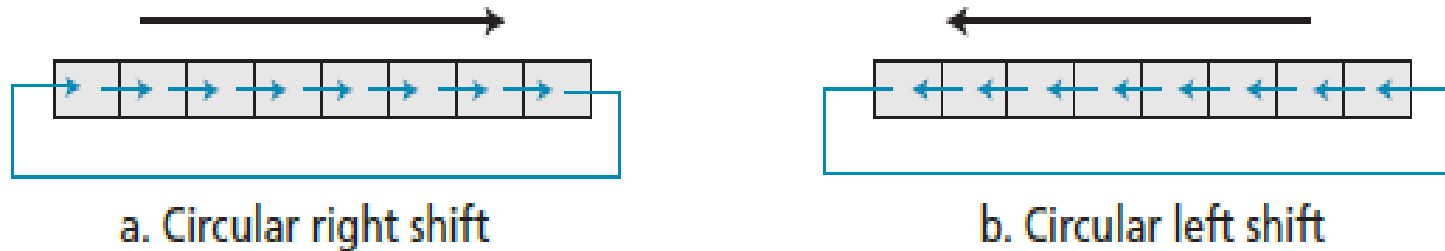
Use a simple left shift operation on the bit pattern 10011000.

Solution

The solution is shown below. The leftmost bit (white in the black background) is lost and a 0 is inserted as the rightmost bit (the bit in color):

←	1	0	0	1	1	0	0	0	Original
	0	0	1	1	0	0	0	0	After shift

Figure 4.4 Circular shift operations



Example 4.11

Use a circular left shift operation on the bit pattern 10011000.

Solution

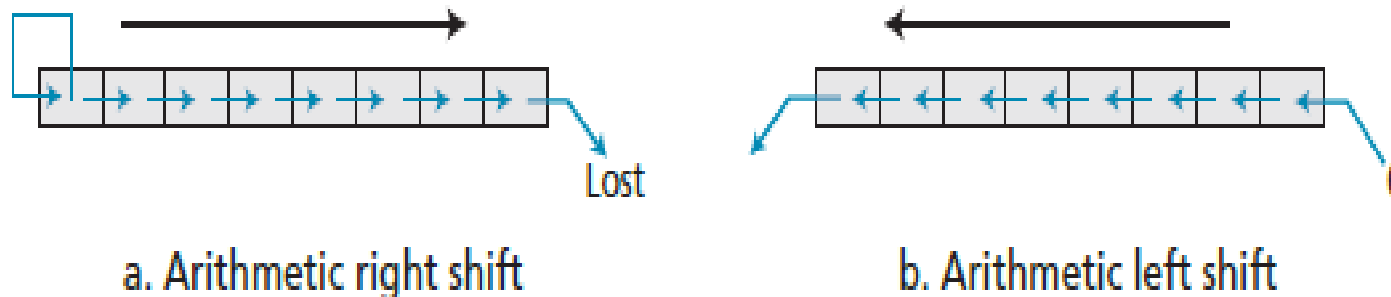
The solution is shown below. The leftmost bit (the white bit in the black background) is circulated and becomes the rightmost bit:

Original	1	0	0	1	1	0	0	0	Circular
After shift	0	0	1	1	0	0	0	1	

4.2.2 Arithmetic shift operations

- Arithmetic shift operations assume that the bit pattern is a signed integer in two's complement format.
- Arithmetic right shift is used to divide an integer by two, while arithmetic left shift is used to multiply an integer by two.

Figure 4.5 *Arithmetic shift operations*



Example 4.12

Use an arithmetic right shift operation on the bit pattern 10011001. The pattern is an integer in two's complement format.

Solution

The solution is shown below. The leftmost bit is retained and also copied to its right neighbor bit (the white bit over the black background). The bit in color is lost:

Arithmetic Right	1	0	0	1	1	0	0	1	Original
	1	1	0	0	1	1	0	0	
									After shift

The original number was -103 and the new number is -52 , which is the result of dividing -103 by 2 truncated to the smaller integer.

Example 4.13

Use an arithmetic left shift operation on the bit pattern 11011001. The pattern is an integer in two's complement format.

Solution

The solution is shown below. The leftmost bit (shown in color) is lost and a 0 (shown as white in the black background) is inserted as the rightmost bit:

Arithmetic Left	1	1	0	1	1	0	0	1	Original
	1	0	1	1	0	0	1	0	
									After shift

The original number was -39 and the new number is -78 . The original number is multiplied by two. The operation is valid because no underflow occurred.

Example 4.14

Use an arithmetic left shift operation on the bit pattern 01111111. The pattern is an integer in two's complement format.

Solution

The solution is shown below. The leftmost bit (in color) is lost and a 0 (white in the black background) is inserted as the rightmost bit:

Arithmetic Left	0	1	1	1	1	1	1	Original
	1	1	1	1	1	1	0	After shift

The original number was 127 and the new number is -2. Here the result is not valid because an overflow has occurred. The expected answer $127 \times 2 = 254$ cannot be represented by an 8-bit pattern.

Example 4.15

Combining logic operations and logical shift operations gives us some tools for manipulating bit patterns. Assume that we have a pattern and we need to use the third bit (from the right) of this pattern in a decision-making process. We want to know if this particular bit is 0 or 1. The following shows how we can find out:

	h	g	f	e	d	c	b	a	Original
	0	h	g	f	e	d	c	b	One right shift
	0	0	h	g	f	e	d	c	Two right shifts
AND	0	0	0	0	0	0	0	1	Mask
	0	0	0	0	0	0	0	c	Result

We shift the pattern two bits to the right so that the target bit moves to the rightmost position. The result is then ANDed with a mask which has one 1 at the rightmost position. The result is a pattern with seven 0s and the target bit at the rightmost position. We can then test the result: if it is an unsigned integer 1, the target bit was 1, whereas if the result is an unsigned integer 0, the target bit was 0.

4-3 ARITHMETIC OPERATIONS

- Arithmetic operations involve **adding, subtracting, multiplying, and dividing**. We can apply these operations to integers and floating-point numbers.

4.3.1 Arithmetic operations on integers

- All arithmetic operations such as addition, subtraction, multiplication, and division can be applied to integers. Although multiplication (division) of integers can be implemented using repeated addition (subtraction), the procedure is not efficient.
- There are more efficient procedures for multiplication and division, such as [Booth multiplication algorithm](#).

Two's complement integers

- When the subtraction operation is encountered, the computer simply changes it to an addition operation, but makes two's complement of the second number.
- In other words:

$$A - B \leftrightarrow A + (\overline{B} + 1)$$

- $\overline{B}$ is the one's complement of B
- $(\overline{B} + 1)$ means the two's complement of B

- We add integers column by column. Table 4.1 shows the sum and carry (C).

Table 4.1 Carry and sum resulting from adding two bits

Column	Sum	Carry
Zero 1s	0	0
One 1	1	0

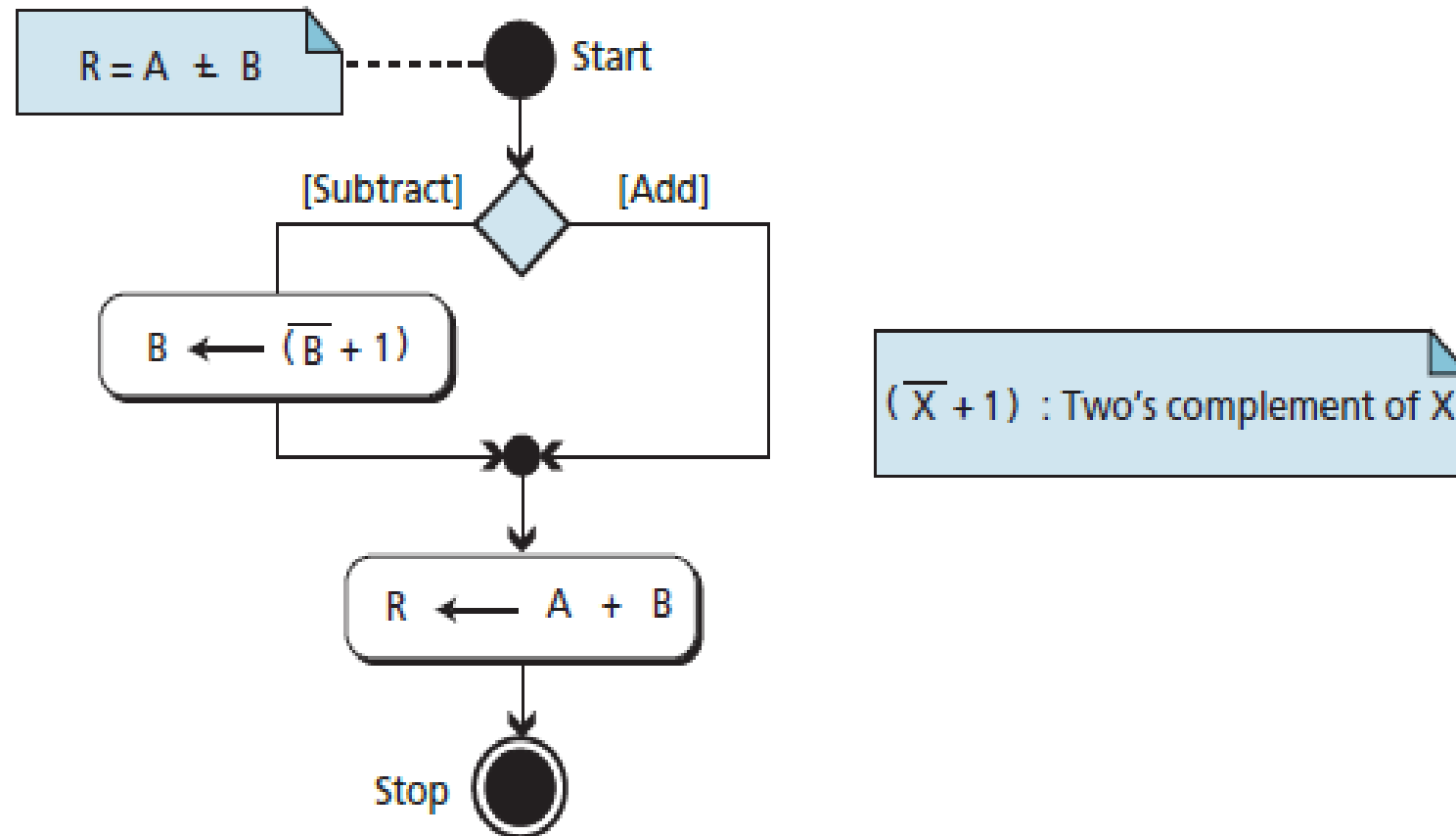
Column	Sum	Carry
Two 1s	0	1
Three 1s	1	1

- We add integers column by column. The following table shows the sum and carry (C).

Table 4.1 Carry and sum resulting from adding two bits and a carry bit

Carry	Sum	+	Carry	Sum	+
0	0	0 0 0	0	1	1 0 0
0	1	0 0 1	1	0	1 0 1
0	1	0 1 0	1	0	1 1 0
1	0	0 1 1	1	1	1 1 1

Figure 4.6 *Addition and subtraction of integers in two's complement format*



Example 4.16

Two integers A and B are stored in two's complement format. Show how B is added to A:

$$A = (00010001)_2 \quad B = (00010110)_2$$

Solution

The operation is adding. A is added to B and the result is stored in R:

				1					Carry
	0	0	0	1	0	0	0	1	A
+	0	0	0	1	0	1	1	0	B
	0	0	1	0	0	1	1	1	R

We check the result in decimal: $(+17) + (+22) = (+39)$.

Example 4.17

Two integers A and B are stored in two's complement format. Show how B is added to A:

$$A = (00011000)_2 \quad B = (11101111)_2$$

Solution

The operation is adding. A is added to B and the result is stored in R. Note that the last carry is discarded because the size of the memory is only 8 bits:

1	1	1	1	1					Carry
	0	0	0	1	1	0	0	0	A
+	1	1	1	0	1	1	1	1	B
	0	0	0	0	0	1	1	1	R

Checking the result in decimal, $(+24) + (-17) = (+7)$.

Example 4.18

Two integers A and B are stored in two's complement format. Show how B is subtracted from A:

$$A = (00011000)_2 \quad B = (11101111)_2$$

Solution

The operation is subtracting. A is added to $(\bar{B} + 1)$ and the result is stored in R:

					1				Carry			
				0	0	0	1	1	0	0	0	A
+				0	0	0	1	0	0	0	1	($\overline{B} + 1$)
				0	0	1	0	1	0	0	1	R

Checking the result in decimal, $(+24) - (-17) = (+41)$.

Example 4.19

Two integers A and B are stored in two's complement format. Show how B is subtracted from A:

$$A = (11011101)_2 \quad B = (00010100)_2$$

Solution

The operation is subtracting. A is added to $(\overline{B} + 1)$ and the result is stored in R:

1	1	1	1	1	1				Carry
	1	1	0	1	1	1	0	1	A
+	1	1	1	0	1	1	0	0	$\overline{B} + 1$
	1	1	0	0	1	0	0	1	R

Checking the result in decimal, $(-35) - (+20) = (-55)$. Note that the last carry is discarded.

Example 4.20

Two integers A and B are stored in two's complement format. Show how B is added to A:

$$A = (01111111)_2 \quad B = (00000011)_2$$

Solution

The operation is adding. A is added to B and the result is stored in R:

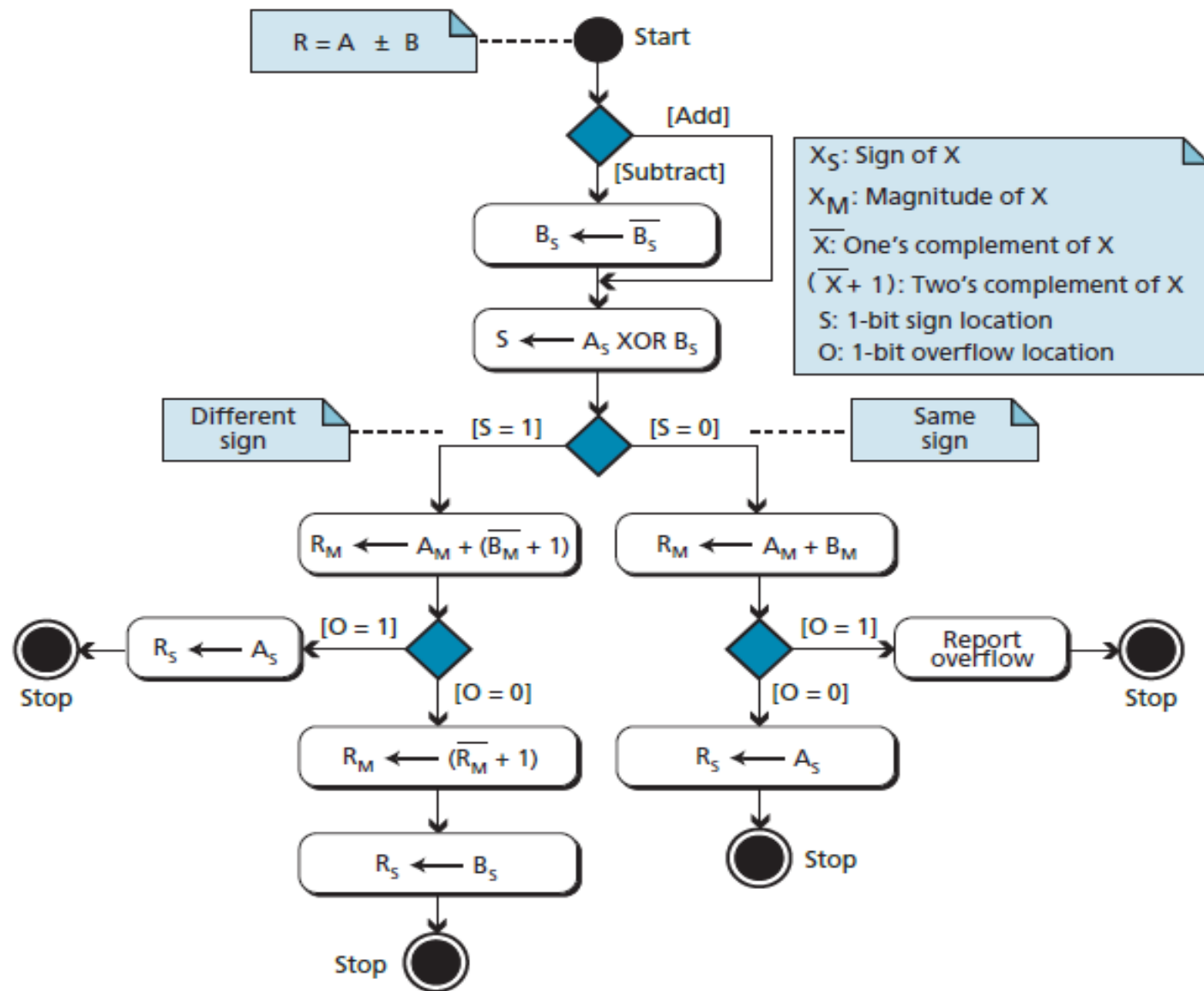
	1	1	1	1	1	1	1	Carry
	0	1	1	1	1	1	1	A
+	0	0	0	0	0	0	1	B
	1	0	0	0	0	0	1	R

We expect the result to be $127 + 3 = 130$, but the answer is -126 . The error is due to overflow, because the expected answer ($+130$) is not in the range -128 to $+127$.

Sign-and-Magnitude integers

- Addition and subtraction for integers in sign-and-magnitude representation looks very complex.
- We have four different combination of signs (two signs, each of two values) for addition, and four different conditions for subtraction.
- This means that we need to consider eight different situations. However, if we first check the signs, we can reduce these cases, as shown in Figure 4.7.

Figure 4.7 Addition and subtraction of integers in sign-and-magnitude format



Example 4.22

Two integers A and B are stored in sign-and-magnitude format. Show how B is added to A.

$$A = (0\ 0010001)_2 \quad B = (1\ 0010110)_2$$

Solution

The operation is adding: the sign of B is not changed. $S = A_s \text{ XOR } B_s = 1$; $R_M = A_M + (B_M + 1)$. Since there is no overflow, we need to take the two's complement of R_M . The sign of R is the sign of B.

No overflow									Carry
A_s	0								A_M
			0	0	1	0	0	0	1
		+							
			1	1	0	1	0	1	0
B_s	1								$(\bar{B}_M + 1)$
			1	1	1	1	0	1	1
									R_M
R_s	1								
			0	0	0	0	1	0	1
									$R_M = (\bar{R}_M + 1)$

Checking the result in decimal, $(+17) + (-22) = (-5)$.

Example 4.23

Two integers A and B are stored in sign-and-magnitude format. Show how B is subtracted from A.

$$A = (1\ 1010001)_2 \quad B = (1\ 0010110)_2$$

Solution

The operation is subtracting: $S_B = S_B$, $S = A_S \text{ XOR } B_S = 1$, $R_M = A_M + (\overline{B_M} + 1)$. Since there is an overflow, the value of R_M is final. The sign of R is the sign of A.

		Overflow →	1						Carry		
A_s	1			1	0	1	0	0	0	1	A_M
B_s	1		+	1	1	0	1	0	1	0	$(\overline{B_M} + 1)$
R_s	1			0	1	1	1	0	1	1	R_M

Checking the result in decimal, $(-81) - (-22) = (-59)$.

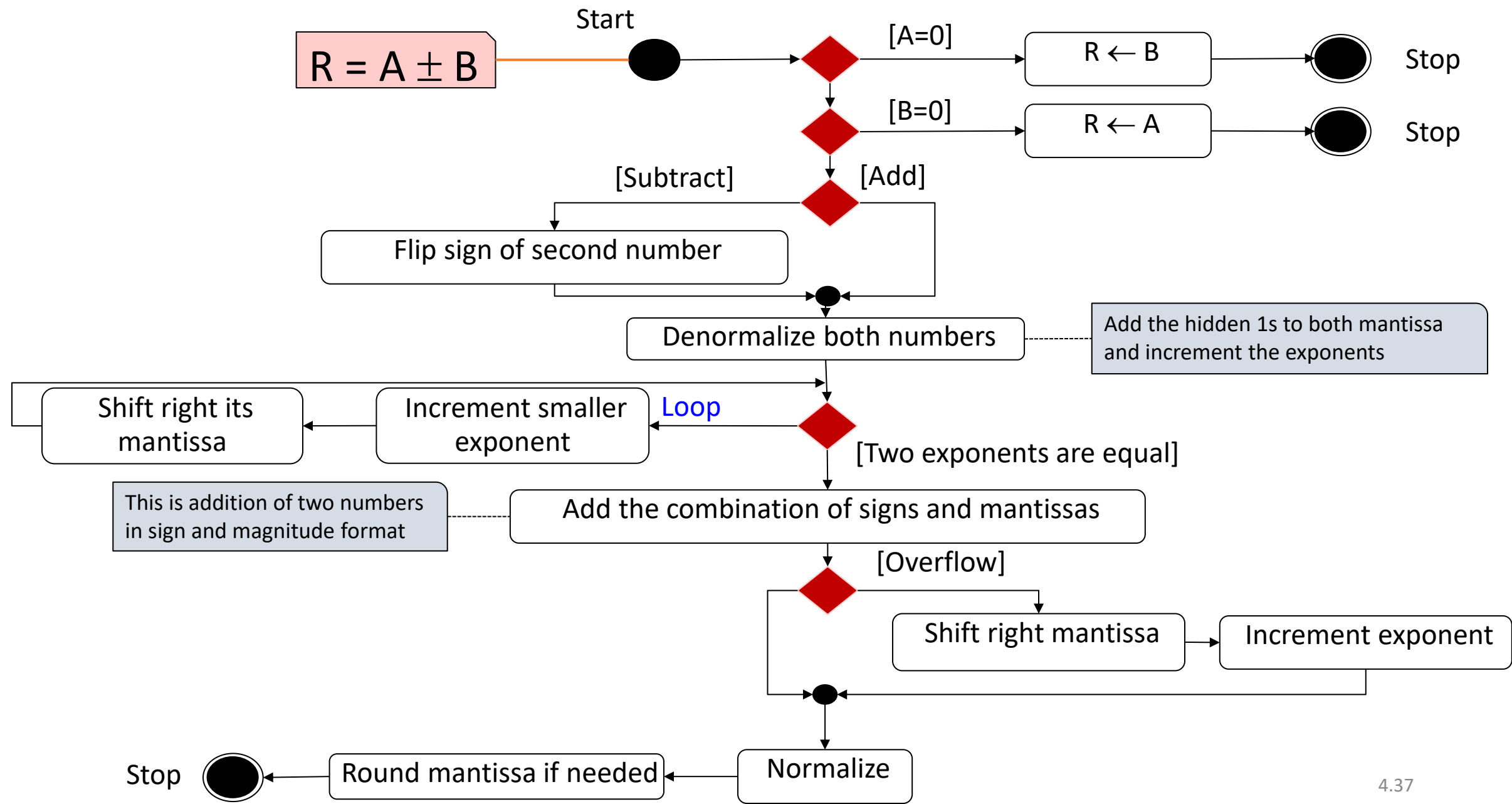
4.3.2 Arithmetic operations on reals

- All arithmetic operations such as addition, subtraction, multiplication, and division can be applied to reals stored in floating-point format.
- Multiplication of two reals involves multiplication of two integers in sign-and-magnitude representation.
- Division of two reals involves division of two integers in sign-and-magnitude representations.
- Since we did not discuss the multiplication or division of integers in sign-and magnitude representation, we will not discuss the multiplication and division of reals, and only show addition and subtractions for reals.

Addition and subtraction of reals

- Addition and subtraction of real numbers stored in floating-point numbers is reduced to addition and subtraction of two integers stored in sign-and-magnitude (combination of sign and mantissa) after the alignment of decimal points.
- Figure J.1 shows a simplified version of the procedure (there are some special cases that we have ignored).

Figure 4.8 Addition and subtraction of reals in floating-point format



Example 4.24

Show how the computer finds the result of $(+5.75) + (+161.875) = (+167.625)$.

Solution

As we saw in Chapter 3, these two numbers are stored in floating-point format, as shown below, but we need to remember that each number has a hidden 1 (which is not stored, but assumed).

	S	E	M
A	0	10000001	011100000000000000000000
B	0	10000110	010000111100000000000000

Now we do sign-and-magnitude addition, treating the sign and the mantissa of each number as one integer stored in sign-and-magnitude representation.

	S	E	Denormalized M
R	0	10000111	101001111010000000000000

There is no overflow in the mantissa, so we normalize.

	S	E	M
R	0	10000110	010011110100000000000000

The mantissa is only 23 bits, no rounding is needed. $E = (10000110)_2 = 134$
 $M = 0100111101$. In other words, the result is $(1.0100111101)_2 \times 2^{134 - 127} = (10100111.101)_2 = 167.625$.

Example 4.25

Show how the computer finds the result of $(+5.75) + (-7.0234375) = -1.2734375$.

Solution

These two numbers can be stored in floating-point format, as shown below:

	S	E	Denormalized M
A	0	10000001	011100000000000000000000
B	1	10000001	111000001100000000000000

Denormalization results in:

	S	E	Denormalized M
A	0	10000010	101110000000000000000000
B	1	10000010	111000001100000000000000

Alignment is not needed (both exponents are the same), so we apply addition operation on the combinations of sign and mantissa. The result is shown below, in which the sign of the result is negative:

	S	E	Denormalized M
R	1	10000010	001010001100000000000000

Now we need to normalize. We decrement the exponent three times and shift the denormalized mantissa to the left three positions:

	S	E	M
R	1	01111111	010001100000000000000000

The mantissa is now 24 bits, so we round it to 23 bits.

	S	E	M
R	1	01111111	010001100000000000000000

The result is $R = -2^{127-127} \times 1.0100011 = -1.2734375$, as expected.

```
#include <iostream>
#include <bitset>
using namespace std;

int main()
{
    int a = 128, b = a >> 2, c = -313, d = c << 1;

    cout << "a = " << bitset<8>(a) << " = " << a << endl;
    cout << "b = " << bitset<8>(b) << " = " << b << endl;
    cout << "c = " << bitset<16>(c) << " = " << c << endl;
    cout << "d = " << bitset<16>(d) << " = " << d << endl;

    return 0;
}
```

output

a = 10000000 = 128

b = 00100000 = 32

c = 1111111011000111 = -313

d = 1111110110001110 = -626

```
#include <iostream>
using namespace std;

void printIntBinary(int v)
{
    size_t L = sizeof(v);
    unsigned char* ptr = (unsigned char *)&v + L-1;

    for(size_t byte = 0; byte < L; ++byte){
        for(unsigned char mask = 0x80; mask != 0; mask >>= 1){
            cout << ((*ptr & mask) ? '1' : '0');
        }
        ptr = --ptr;
        cout << ' ';
    }
    cout << endl;
}
```

output

```
int main()
{
    int n = 33;
    printIntBinary(n);
}

00000000 00000000 00000000 00100001
```

```

#include <iostream>
using namespace std;

template<typename T>
void printBinary(const T& v, int endian = 0)
{
    size_t L = sizeof(v);

    unsigned char* ptr = endian ? ((unsigned char*)&v) : ((unsigned char*)&v)+L-1 ;
    for(size_t byte = 0; byte < L; ++byte){
        for(unsigned char mask = 0x80; mask != 0; mask >>= 1){
            cout << ((*ptr & mask) ? '1' : '0');
        }
        ptr = endian ? (++ptr) : (--ptr);
        cout << ' ';
    }
    cout << endl;
}

```

```

int main()
{
    char a = 33;
    short b = 33;
    int c = 33;
    float f = 3.14;
    printBinary(a);
    printBinary(b);
    printBinary(c);
    printBinary(f);
}

00100001
00000000 00100001
00000000 00000000 00000000 00100001
01000000 01001000 11110101 11000011

```

```

int main()
{
    char a = 33;
    short b = 33;
    int c = 33;
    float f = 3.14;
    printBinary(a,1);
    printBinary(b,1);
    printBinary(c,1);
    printBinary(f,1);
}

00100001
00100001 00000000
00100001 00000000 00000000 00000000
11000011 11110101 01001000 01000000

```

Exercises

- P. 98 4.5.3 Problems
 - p4-7, p4-8, p4-12, p4-14, p4-19, p4-20, p4-21